

CBIO204 – Fundamentals of Databases

Biotechnology Database Project

Biobank and Biospecimen Management System

Name: Nadien Ahmed Shawky

ID: 221002162

Dr/ Muhammed Elsayeh

1. Problem Statement, Scope, Assumptions, and Business Rules

The project implements a relational database for a biobank that stores and manages biospecimens collected from donors for biomedical research. The system tracks donors and consent, collection events, sample types, biospecimens, storage positions, aliquots, researchers, test requests, sample usage, and test results. The design follows the project guideline requirement for a realistic biotechnology database with strong entities, one-to-many relationships, a many-to-many relationship resolved through an associative table, constraints, SQL operations, views, and an advanced database object.

In scope:

- Donor registration and consent tracking.
- Recording biospecimen collection events and sample classification.
- Tracking specimen status, quality, volume, aliquots, and freezer locations.
- Managing researchers and their test requests.
- Recording sample usage and laboratory test results.
- Enforcing integrity rules with primary keys, foreign keys, CHECK constraints, UNIQUE constraints, indexes, and a trigger.

Assumptions:

- All donor identifiers are synthetic and anonymized for the project.
- A donor may have multiple collection events and multiple consent records.
- A collection event can produce multiple biospecimens.
- A biospecimen can be divided into multiple aliquots.
- An aliquot can be associated with one storage position at a time.
- A researcher can submit multiple test requests.
- A test request can use multiple aliquots, and an aliquot can support multiple requests; SAMPLE_USAGE resolves this many-to-many relationship.
- The implementation targets PostgreSQL 15+.

Major business rules:

- Donor code, researcher code, sample code, aliquot code, and researcher email are unique.
- A consent expiry date cannot precede its consent date.
- Specimen volume and aliquot volume must be positive.
- Storage temperatures must be at or below 8°C for this project.
- A sample usage quantity must be positive and cannot exceed the aliquot's available volume.
- A valid sample usage reduces the remaining aliquot volume; when it reaches zero, the aliquot becomes Depleted.
- Foreign keys prevent orphan records and restrict deletion of referenced core records.

2. Entities, Attributes, and Keys

DONOR — donor_id PK; donor_code UQ; first_name; last_name; date_of_birth; sex; email UQ; phone; registration_date; status

CONSENT — consent_id PK; donor_id FK; consent_type; consent_date; expiry_date; status; version_no

SAMPLE_TYPE — sample_type_id PK; type_name UQ; description

COLLECTION_EVENT — collection_event_id PK; donor_id FK; collection_date; collection_site; collector_name; protocol_code; notes

BIOSPECIMEN — sample_id PK; sample_code UQ; collection_event_id FK; sample_type_id FK; volume_ml; collection_time; quality_status; current_status

STORAGE_LOCATION — location_id PK; facility; room_code; freezer_code; shelf_no; box_code; position_no; temperature_c

ALQUOT — aliquot_id PK; sample_id FK; aliquot_code UQ; volume_ml; location_id FK; created_at; status

RESEARCHER — researcher_id PK; researcher_code UQ; full_name; email UQ; department; active_flag

TEST_REQUEST — test_request_id PK; researcher_id FK; request_date; purpose; priority; status

SAMPLE_USAGE — usage_id PK; test_request_id FK; aliquot_id FK; usage_date; quantity_used_ml; result_summary

SAMPLE_TEST — sample_test_id PK; test_request_id FK; sample_id FK; test_name; test_date; result_value; result_unit; result_status

3. ER Diagram and Relationship Design

The final ER diagram is included in the project materials. The main relationships are:

- DONOR 1:N CONSENT — each consent record belongs to one donor; a donor can have multiple consent records.
- DONOR 1:N COLLECTION_EVENT — a donor may participate in multiple collection events.
- COLLECTION_EVENT 1:N BIOSPECIMEN — one collection event may produce multiple biospecimens.
- SAMPLE_TYPE 1:N BIOSPECIMEN — each biospecimen has one type; a type can classify many biospecimens.
- BIOSPECIMEN 1:N ALQUOT — one biospecimen may be divided into multiple aliquots.
- STORAGE_LOCATION 1:N ALQUOT — a location can hold multiple aliquots; location is nullable to support temporary unassigned aliquots.
- RESEARCHER 1:N TEST_REQUEST — a researcher can submit many requests.
- TEST_REQUEST M:N ALQUOT — resolved by SAMPLE_USAGE.
- TEST_REQUEST 1:N SAMPLE_TEST and BIOSPECIMEN 1:N SAMPLE_TEST — tests connect requested work with the sample being tested.

Participation is mandatory on the child side for the core foreign keys; ALQUOT.location_id is optional.

Trigger – T08 usage exceeding aliquot volume is rejected

test_case	attempted_quantity_ml	result
T08 – usage exceeds aliquot volume	9999	REJECTED – Usage quantity exceeds available aliquot volume

4. Relational Schema

The ER model is mapped into eleven normalized relations. Primary keys identify individual rows. Foreign keys preserve referential integrity. UNIQUE constraints represent alternate candidate keys such as donor_code, sample_code, aliquot_code, researcher_code, and researcher_email.

- donor(donor_id PK, donor_code UQ, first_name, last_name, date_of_birth, sex, email UQ, phone, registration_date, status)
- consent(consent_id PK, donor_id FK→donor, consent_type, consent_date, expiry_date, status, version_no, UQ(donor_id, consent_type, version_no))
- sample_type(sample_type_id PK, type_name UQ, description)
- collection_event(collection_event_id PK, donor_id FK→donor, collection_date, collection_site, collector_name, protocol_code, notes)
- biospecimen(sample_id PK, sample_code UQ, collection_event_id FK→collection_event, sample_type_id FK→sample_type, volume_ml, collection_time, quality_status, current_status)
- storage_location(location_id PK, facility, room_code, freezer_code, shelf_no, box_code, position_no, temperature_c, UQ(freezer_code, shelf_no, box_code, position_no))
- aliquot(aliquot_id PK, sample_id FK→biospecimen, aliquot_code UQ, volume_ml, location_id FK→storage_location, created_at, status)
- researcher(researcher_id PK, researcher_code UQ, full_name, email UQ, department, active_flag)
- test_request(test_request_id PK, researcher_id FK→researcher, request_date, purpose, priority, status)
- sample_usage(usage_id PK, test_request_id FK→test_request, aliquot_id FK→aliquot, usage_date, quantity_used_ml, result_summary, UQ(test_request_id, aliquot_id))
- sample_test(sample_test_id PK, test_request_id FK→test_request, sample_id FK→biospecimen, test_name, test_date, result_value, result_unit, result_status, UQ(test_request_id, sample_id, test_name))

5. Normalization to 3NF

The design satisfies Third Normal Form. Each table represents one subject or relationship and attributes depend on the key of that table.

- 1NF: all attributes are atomic; repeating groups such as multiple samples, aliquots, or tests are stored as separate rows.
- 2NF: non-key attributes in each relation depend on the whole primary key. Associative data such as quantity_used_ml belongs to SAMPLE_USAGE rather than to either TEST_REQUEST or ALIQUOT alone.
- 3NF: transitive dependencies are removed. For example, sample type details are stored in SAMPLE_TYPE instead of repeated in BIOSPECIMEN, and researcher department/contact details are stored in RESEARCHER rather than TEST_REQUEST.
- The M:N relationship between TEST_REQUEST and ALIQUOT is resolved by SAMPLE_USAGE, which has its own primary key and relationship-specific attributes.
- Storage coordinates are kept in STORAGE_LOCATION so the same location information is not repeated for every aliquot.

6. Implementation Details

PostgreSQL 15+ is used. The implementation is separated into scripts so that the database can be reproduced from a clean database.

- create_tables.sql: schema, 11 tables, PK/FK constraints, CHECK constraints, UNIQUE constraints, and indexes.
- load_data.sql: synthetic test data with 10 records in each main table.
- queries.sql: retrieval, joins, aggregation, subqueries, INSERT, UPDATE, DELETE, and business queries.
- views.sql: v_sample_inventory and v_research_activity.
- triggers_procedures.sql: fn_validate_sample_usage trigger function, trigger, and fn_donor_sample_count function.

7. Views and Advanced Database Object

View 1 — `v_sample_inventory`: provides an inventory-oriented summary of each biospecimen, including donor, sample type, current status, aliquot count, and total aliquot volume.

View 2 — `v_research_activity`: summarizes each researcher's test requests, sample usage count, and total volume used.

Trigger — `trg_validate_sample_usage`: before a `SAMPLE_USAGE` row is inserted, the trigger checks that the requested quantity does not exceed the aliquot's available volume. A valid usage reduces the aliquot volume and changes the status to Used or Depleted.

Function — `fn_donor_sample_count(donor_id)`: returns the number of biospecimens associated with a donor.

8. Sample SQL Operations

Retrieval

```
SELECT sample_code, volume_ml FROM biospecimen WHERE current_status = 'Available';
```

Join

```
SELECT b.sample_code, d.donor_code, st.type_name FROM biospecimen b JOIN collection_event ce ON
ce.collection_event_id=b.collection_event_id JOIN donor d ON d.donor_id=ce.donor_id JOIN sample_type
st ON st.sample_type_id=b.sample_type_id;
```

Aggregation

```
SELECT st.type_name, COUNT(*) FROM sample_type st LEFT JOIN biospecimen b ON
b.sample_type_id=st.sample_type_id GROUP BY st.type_name;
```

Subquery

```
SELECT donor_code FROM donor WHERE donor_id IN (SELECT donor_id FROM collection_event);
```

Views

```
SELECT * FROM v_sample_inventory;

SELECT * FROM v_research_activity;
```

Function

```
SELECT fn_donor_sample_count(1);
```

The deployed SQLite demo does not support `CREATE FUNCTION` syntax, so the function result was produced with the equivalent join/aggregate query; `fn_donor_sample_count` is implemented as a PL/pgSQL function in `triggers_procedures.sql` for the PostgreSQL target.

View — `v_sample_inventory`

sample_code	donor_code	type_name	current_status	aliquot_count	total_aliquot_volume
S001	D001	Whole Blood	Available	1	4.0
S002	D002	Plasma	Available	1	3.5
S003	D003	Serum	Available	1	5.0
S004	D004	Urine	Available	1	3.0
S005	D005	Saliva	Available	1	4.5
S006	D006	Tissue	Available	1	2.5
S007	D007	DNA	Available	1	2.0
S008	D008	RNA	Available	1	3.0
S009	D009	PBMC	Available	1	4.0
S010	D010	CSF	Available	1	2.5

researcher_code	full_name	request_count	usage_count	total_volume_used
R001	Dr. Researcher 1	1	1	1.6
R002	Dr. Researcher 2	1	1	1.4
R003	Dr. Researcher 3	1	1	2.0
R004	Dr. Researcher 4	1	1	1.2
R005	Dr. Researcher 5	1	1	1.8
R006	Dr. Researcher 6	1	1	1.0
R007	Dr. Researcher 7	1	1	0.8
R008	Dr. Researcher 8	1	1	1.2
R009	Dr. Researcher 9	1	1	1.6
R010	Dr. Researcher 10	1	1	1.0

Function — fn_donor_sample_count(1)

donor_code	sample_count
D001	1

9. Testing Plan and Expected Results

T01 — Insert valid donor — Row is inserted successfully. — Pass expected

T02 — Duplicate donor_code — UNIQUE constraint rejects duplicate. — Pass expected

T03 — Invalid donor sex — CHECK constraint rejects value outside M/F/O. — Pass expected

T04 — Invalid consent expiry — CHECK constraint rejects expiry before consent date. — Pass expected

T05 — Invalid foreign key — Child row referencing a nonexistent donor is rejected. — Pass expected

T06 — Negative aliquot volume — CHECK constraint rejects non-positive volume. — Pass expected

T07 — Valid sample usage — Trigger reduces aliquot volume and sets status appropriately. — Pass expected

T08 — Usage greater than aliquot volume — Trigger raises an exception and prevents insertion. — Pass expected

T09 — View execution — Both views return rows from the loaded data. — Pass expected

T10 — Clean execution — Scripts execute in the documented order on a clean database. — Pass expected

The trigger and constraint checks in the plan above were executed against the deployed database; representative evidence is included in the source report.

10. Limitations and Future Improvements

- The dataset is synthetic and intended for academic demonstration.
- Authentication and role-based database permissions are outside the current project scope.
- A production system would require audit history for every change and stronger privacy controls.
- A production biobank could add freezer monitoring, temperature alerts, shipment tracking, barcode scanning, and laboratory instrument integration.
- A user interface can be added for the optional bonus marks, provided it connects to the database and performs validated CRUD operations.

11. Conclusion

The project delivers a normalized relational database for a realistic biobank workflow. It covers requirements analysis, conceptual design, relational mapping, 3NF normalization, PostgreSQL implementation, synthetic test data, SQL operations, two views, a trigger, a function, integrity constraints, indexes, and a testing plan. The structure is organized to match the required repository and report sections in the CBIO204 project guidelines.

12. References

CBIO204 Fundamentals of Databases — Biotechnology Database Project Guidelines, provided course document.

PostgreSQL Documentation, PostgreSQL Global Development Group. Consult the official PostgreSQL documentation for syntax and DBMS behavior.